Laboratory Work 3

A Basic Informative Program:

Data Types, Variables and Arithmetic

Program structure · Variables · `int` `float` `double` `char` `bool` · `cin`/`cout` · `/` and `%` · `sizeof`

Week: 3 — first session
Duration: 2 hours 15 minutes
Format: guided theory, then individual practice
Tools: computer with CLion (or any C++17 compiler); pen for Sections A–G
Prerequisite: Lab 2, Section K (pseudocode → C++)

Name:

Group:

Date:

Learning outcomes

By the end of this lab you should be able to:

- **LO1.** Name every part of a minimal C++ program and explain what each line does.
- **LO2.** Declare and initialise variables of the types `int`, `float`, `double`, `char` and `bool`, following the identifier rules, and choose the right type for a quantity.
- **LO3.** Read values with `cin` and print them with `cout`.
- **LO4.** Evaluate arithmetic expressions using precedence, integer division `/` and remainder `%`.
- **LO5.** Translate a short algorithm from Lab 2 into a complete C++ program: input, process, output.
- **LO6.** Write, compile and run a small informative program that reads data and prints a labelled report.

This lab covers Tutorial 1 of Week 3 in the course syllabus (*Basic informative program*) and the C++ half of the Week 2 lecture: program structure, syntax, input/output streams, data types and variables. Output formatting with `<iomanip>` (`setw`, `setprecision`) is deliberately left out and will be covered in a later lab. Lab 4 continues with type conversions and `if`/`else`.

How this lab is organised

Work through the document in order. **Part I** explains each idea and gives a worked example; after most ideas there is a short **Quick check** that you solve immediately, before moving on. **Part II** is the graded practice set. Sections A–G are done *on paper, without a computer* — predicting what code does is the skill being tested. Sections H–K are done in **CLion**. Do not start Part II until your instructor says so.

Block	Content		Time
–	Introduction and learning outcomes		5 min
PART I — THEORY AND WORKED EXAMPLES			40 min
§1	Anatomy of a C++ program		5 min
§2	Variables and identifiers		5 min
§3	Fundamental data types		7 min
§4	Input and output streams		6 min
§5	Arithmetic operators: `/`, `%`, precedence		9 min
§6	From pseudocode to a complete program		6 min
§7	Common mistakes		2 min
–	*Short break*		5 min
PART II — PRACTICE			80 min
A–B	Identifiers; choosing a data type	*paper*	9 min
C	Predict the output: arithmetic	*paper*	10 min
D	Trace the variables	*paper*	8 min
E	Find the mistake	*paper*	6 min
F–G	True/false and multiple choice	*paper*	8 min
H	A basic informative program	*CLion*	10 min
I	Integer calculator	*CLion*	8 min
J	Time breakdown with `/` and `%`	*CLion*	8 min
K	Shop bill: VAT and change	*CLion*	13 min
L	Bonus challenges *(not timed)*		—
–	Submission checklist and wrap-up		5 min
Total			**2 h 15 min**

How your work is checked. Nothing is uploaded — everything is checked here, during the lab. Write the answers to Sections A–G on this sheet. Do Sections H–K in CLion, one `.cpp` file per task, and run each program with the test input given in the task. **When you have finished, raise your hand** and show the TA your answer sheet and each program running. The TA marks the score table on the last page.

PART I — Theory and worked examples

1 Anatomy of a C++ program

In Lab 2 you wrote algorithms in pseudocode. C++ says the same things, but under strict **syntax** rules that the compiler checks letter by letter. Here is the smallest useful program.

```
#include <iostream>      // 1
using namespace std;     // 2

int main() {             // 3
    // print a greeting    4
    cout << "Hello, CS102!" << endl;  // 5
    return 0;            // 6
}                        // 7
```

1. `#include` copies in a *library*; `<iostream>` gives `cin` and `cout`.
2. Lets you write `cout` instead of `std::cout`.
3. Every program starts running at `main`. The braces `{ }` mark its body.
4. A comment — ignored by the compiler, written for people.
5. A *statement*. Every statement ends with a semicolon `;`.
6. Tells the operating system the program finished normally.
7. Closes `main`.

In CLion, **Build** runs the compiler, which turns the source into machine code; **Run** executes it. If the compiler finds a **syntax error** (a missing `;`, a misspelt keyword) no program is produced at all. A program that builds can still contain a **logic error** — the compiler checks *grammar*, not *meaning*.

Quick check 1. What goes wrong in each case? (a) line 5 has no `;` at the end; (b) line 1 is deleted; (c) `main` is spelt `Main`.

2 Variables and identifiers

Variable

A **variable** is a named memory location that holds a value of one fixed **type**. It must be **declared** before use: `type name = initial_value;`

```
int    students = 24;       // declaration + initialisation
double gpa;                 // declared, NOT initialised: holds garbage until assigned
gpa = 3.45;                 // assignment (Lab 2: SET gpa = 3.45)
const double VAT = 0.12;    // a constant: any later assignment is a compile error
```

Identifier rules (the names you invent):

- letters, digits and underscore _ only — no spaces, no `-`, no `$`, no `@`;
- must not start with a digit;
- must not be a **keyword** (`int`, `double`, `return`, `if`, `while`, ...);
- **case-sensitive**: `total`, `Total` and `TOTAL` are three different variables.

Convention in this course: `camelCase` for variables (`totalPrice`), `UPPER_CASE` for constants (`MAX_SCORE`). Names should say what the value *means*: `seconds`, not `s1`.

Quick check 2. Which are legal? `score2` `2score` `max_value` `max-value` `Return`

3 Fundamental data types

The type decides three things: which values fit, how many bytes they take, and which operations mean what. The sizes below are what every compiler you will meet in CS102 uses.

Type	Bytes	Literal examples	Holds
`int`	4	`42`, `-7`, `0`	whole numbers, about ±2.1 billion ($2^{31} - 1 = 2\,147\,483\,647$)
`long long`	8	`8100000000LL`	whole numbers up to about 9.2×10^{18}
`float`	4	`3.14f`	real numbers, about 7 significant digits
`double`	8	`3.14`, `2.5e-3`	real numbers, about 15 significant digits — **the default choice**
`char`	1	`'A'`, `'7'`, `'\n'`	one character, stored as its ASCII code
`bool`	1	`true`, `false`	a truth value
`string`	—	`"Aziza"`	text (needs `#include <string>`)

`sizeof` gives the number of bytes: `sizeof(int)` is 4, `sizeof(double)` is 8.

Worked example 3.1 — a `char` is a small number

Recall Lab 1: memory holds only bits. A `char` holds the ASCII *code* of the character.

```
char c = 'A';
cout << c << endl;              // A      printed as a character
cout << c + 1 << endl;          // 66     'A' is 65; c + 1 is an int
cout << char(c + 1) << endl;    // B      converted back to a character
cout << true << " " << false;   // 1 0    bool prints as a number by default
```

Single quotes make a `char`: `'A'`. Double quotes make text: `"A"`. They are different types, and `char grade = "A";` does not compile. Also note `'7'` is the character with code 55, not the number 7.

`float` is less precise than it looks. `float f = 0.1f;` printed with 10 significant digits shows `0.1000000015`. Binary cannot store 0.1 exactly (Lab 1 again), and a `float` keeps only about 7 correct digits. Use `double` unless you have a reason not to.

Quick check 3. Choose a type for: the number of seconds in a day; your height in metres; whether a student passed; the grade letter.

4 Input and output streams

`cout` ("see-out") sends values to the screen with the **insertion** operator `<<`. `cin` ("see-in") reads from the keyboard with the **extraction** operator `>>`. The arrows point the way the data flows.

```
int age;
double height;
cout << "Age and height: ";
cin >> age >> height;          // chain: two values
cout << "Age " << age << ", height "
     << height << " m" << endl;
cout << "Tab:\tNew line:\n";   // escape sequences
```

`cin >> x` skips spaces, tabs and newlines, then reads *one* value of `x`'s type. So the user may type `19 1.72` on one line or on two lines — both work. `endl` and `'\n'` both end the line. Common escape sequences: `\n` new line, `\t` tab, `\"` a quote, `\\` a backslash.

`cin >>` reads one word. If the user types `Aziza Karimova` into `cin >> name;`, only `Aziza` is stored; `Karimova` stays waiting for the next `>>`. Reading a whole line needs `getline(cin, name)`, which you will use later. In this lab, names are single words.

Quick check 4. The user types `5 2.5 7`. What does `int a; double b; int c; cin >> a >> b >> c;` store?

5 Arithmetic operators

Op	Meaning	Example	Value
`+ - *`	add, subtract, multiply	`7 * 3`	`21`
`/`	division — **integer** if both sides are integers	`17 / 5` `17 / 5.0`	`3` `3.4`
`%`	remainder (**integers only**)	`17 % 5`	`2`

Integer division

When *both* operands of `/` are integers, the result is an integer: the fractional part is **thrown away** (truncated towards zero), not rounded. `7 / 2` is `3`; `-7 / 2` is `-3`; `1 / 2` is `0`. The remainder satisfies `(a / b) * b + a % b == a`, so `-17 % 5` is `-2`. If at least one operand is a `double`, the division is real: `7 / 2.0` is `3.5`.

Precedence. `*`, `/`, `%` bind tighter than `+` and `-`; operators of equal precedence are applied **left to right**. Parentheses override everything.

`2 + 3 * 4 - 6 / 4` $= 2 + 12 - 1 = 13$ $\qquad$ `10 - 4 - 3` $= (10 - 4) - 3 = 3$

Worked example 5.1 — taking a number apart with / and %

`n / 10` removes the last digit; `n % 10` *is* the last digit. This is the repeated-division method from Lab 1, and the `DIV`/`MOD` of Lab 2, now in C++.

```
int n = 4827;
int units = n % 10;          // 7
int tens  = n / 10 % 10;     // 482 % 10 = 2   (left to right: / first, then %)
int hund  = n / 100 % 10;    // 48 % 10  = 8
int thou  = n / 1000;        // 4
```

The same idea converts units: `3725` seconds is `3725 / 60 = 62` minutes and `3725 % 60 = 5` seconds; then `62 / 60 = 1` hour and `62 % 60 = 2` minutes.

Shorthand operators. `x += 3` means `x = x + 3`; likewise `-=`, `*=`, `/=`, `%=`. `x++` and `++x` both add one to `x`; inside a larger expression, `x++` gives the *old* value and `++x` the *new* one:

```
int n = 5;
int m = n++ + 2;     // uses 5, then n becomes 6   ->  m = 7
int p = ++n * 2;     // n becomes 7 first, then 7*2 ->  p = 14
```

Quick check 5. Evaluate: `20 / 6`, `20 % 6`, `20 / 6.0` (to 2 d.p.), `1 / 3 * 3`, `7 - 2 * 3 + 1`.

6 From pseudocode to a complete program

Every program in this lab has the same three-part shape — the *input, process, output* of Lab 2's first stage. Here is Lab 2, Worked example 2.1 (the average of three numbers), line by line in C++.

Worked example 6.1 — average of three numbers

Lab 2 pseudocode

```
START
INPUT n1, n2, n3
SET sum = n1 + n2 + n3
SET average = sum / 3
PRINT average
END
```

C++

```cpp
#include <iostream>
using namespace std;

int main() {
    double n1, n2, n3;                    // declare
    cout << "Enter three numbers: ";      // prompt
    cin >> n1 >> n2 >> n3;                // INPUT
    double sum = n1 + n2 + n3;            // SET
    double average = sum / 3;             // SET
    cout << "Average: " << average << endl;  // PRINT
    return 0;
}
```

Three things C++ adds: every variable gets a **type** before use; the user sees a **prompt** so they know what to type; and the output has a **label**, so `8.33333` means something. For inputs `7 8 10` the program prints `Average: 8.33333`.

How `cout` prints a `double` by default. It shows at most 6 significant digits and drops trailing zeros: `25.0 / 3` prints `8.33333`, `2.50` prints `2.5`, `12.0` prints `12`. That is all you need for this lab; controlling the number of decimals comes later in the course.

Quick check 6. What does `cout << 10.0 / 4 << " " << 10 / 4 << " " << 1.0 / 3;` print?

7 Common mistakes — and how to catch them

Mistake	Symptom and fix
`double avg = (a + b) / 2;` with `int a, b`	Integer division happens *before* the value reaches `avg`: 7 and 8 give 7, not 7.5. Write `/ 2.0`.
Missing `;` or `}`	Build fails; the error is often reported on the *next* line. Look one line up.
`char c = "A";`	Double quotes are text. A `char` needs single quotes.
Using a variable before giving it a value	Prints garbage that changes from run to run. Initialise every variable.
`cin << x;` or `cout >> x;`	Build fails: the arrows point the wrong way. `cin >>` into a variable, `cout <<` out to the screen.
No prompt before `cin`	The program sits waiting and the user does not know what to type. Print a prompt first.

PART II — Practice

A Warm-up: legal identifiers

[4 min] (5 points)

Write **OK** if the name is a legal identifier, otherwise give the reason. ($\frac{1}{2}$ each)

1. `totalSum` _____

2. `2ndPlace` _____

3. `my_var` _____

4. `double` _____

5. `student age` _____

6. `_count` _____

7. `price-tax` _____

8. `Int` _____

9. `x2` _____

10. `return` _____

B Choosing a data type

[5 min] (8 points)

For each quantity, choose the most suitable type and write a declaration with a sensible value. (1 each)

Quantity	Type	Declaration
Number of students in the group		
A student's GPA		
First letter of your surname		
Is the library open?		
Temperature in Tashkent, °C		
Seconds in a (non-leap) year		
The value of π, never changing		
World population, about 8.1 billion		

C Predict the output: arithmetic

[10 min] (12 points)

Without a computer, write what each numbered line prints. (1 per line)

```
1  int a = 17, b = 5;
2  cout << a / b << endl;                  // 1
3  cout << a % b << endl;                  // 2
4  cout << a / b * b + a % b << endl;      // 3
5  cout << 17 / 5.0 << endl;               // 4
6  cout << -17 / 5 << endl;                // 5
7  cout << -17 % 5 << endl;                // 6
8  cout << 2 + 3 * 4 - 6 / 4 << endl;      // 7
9  cout << (2 + 3) * (4 - 6) / 4 << endl;  // 8
10 cout << 10 - 4 - 3 << endl;             // 9
11 cout << 1 / 2 * 10 << endl;             // 10
12 int x = 5;
13 x += 3;
14 x *= 2;
15 x %= 7;
16 cout << x << endl;                      // 11
17 int n = 5;
18 int m = n++ + 2;
19 int p = ++n * 2;
20 cout << n << " " << m << " " << p << endl;  // 12
```

Line	Output
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	

D Trace the variables

[8 min] (12 points)

Trace this fragment exactly as in Lab 2, Section G: after each line, write the value of *both* variables. (2 per row)

```
1  int a = 7, b = 3;
2  a = a + b;          // 1
3  b = a - b;          // 2
4  a = a - b;          // 3
5  b *= 2;             // 4
6  a = b % a + a;      // 5
7  b = b / 4 - a;      // 6
```

Line	a	b	Working
start	7	3	initial values
1			
2			
3			
4			
5			
6			

What did lines 1–3 do to the original values 7 and 3?

E Find the mistake

[6 min] (8 points)

Each fragment has exactly one fault. Say whether it is a **syntax** error (the build fails) or a **logic** error (it runs but is wrong), and write the corrected line.

a) (2)

```
1  int main() {
2      cout << "Result: " << 42 << endl
3      return 0;
4  }
```

b) (2)

```
1  int a = 7, b = 8, c = 10;
2  double avg = (a + b + c) / 3;     // expected 8.33...
```

c) (2)

```
1  char grade = "A";
```

d) (2) Intended: read the user's age.

```
1  int age;
2  cin << age;
```

F True or false

[4 min] (7 points)

Mark each statement **T** or **F**. For every **F**, write one short line saying why.

1. Every C++ statement ends with a semicolon.

2. `Total` and `total` are the same variable.

3. `7 / 2` evaluates to `3.5`.

4. `sizeof(double)` is larger than `sizeof(float)`.

5. `cin >> name;` reads a whole line, including spaces.

6. `'5'` and `5` are the same value. _____

7. The `%` operator can only be used with integer operands. _____

G Multiple choice

[4 min] (7 points)

Circle one answer per question.

1. Which header provides `cin` and `cout`?
(a) `<iostream>` (b) `<cmath>`
(c) `<string>` (d) `<main>`

2. The value of `25 % 7` is
(a) 3 (b) 3.57 (c) 4 (d) 18

3. Which declaration is **invalid**?
(a) `int n = 0;` (b) `double d = 1e3;`
(c) `char c = 'xy';` (d) `bool b = false;`

4. `cout << 5 / 2 * 2;` prints
(a) 5 (b) 4 (c) 1 (d) 5.0

5. `cout << 7.0 / 2;` prints
(a) `3` (b) `3.5` (c) `3.50` (d) `4`

6. The operator used with `cin` is
(a) `<<` (b) `>>` (c) `->` (d) `::`

7. Which type is best for storing `3.75`?
(a) `int` (b) `char` (c) `bool` (d) `double`

H A basic informative program

[10 min] (12 points)

Task H — one `.cpp` file

Write a program that asks for a student's **first name** (`string`), **age** (`int`), **height** in metres (`float`), **group letter** (`char`) and whether they are **full-time** (`bool`, entered as `1` or `0`), then prints an information card:

- every value on its own line with a label, exactly as below;
- one *computed* line: the age in months;
- a last line showing the number of bytes each of the four numeric/character variables occupies, using `sizeof`.

Test input: `Aziza 19 1.68 B 1`

```
--- Student card ---
Name:          Aziza
Age:           19
Height:        1.68 m
Group:         B
Full-time:     1
Age in months: 228
Bytes: int 4, float 4, char 1, bool 1
```

Your output matches? ☐ yes ☐ no — what differs: _____

I Integer calculator

[8 min] (8 points)

Task I — one `.cpp` file

Read two integers a and b (assume $b \neq 0$). Print their sum, difference, product, integer quotient, remainder, and the **exact** quotient. Do not declare any `double` variable — make the division real inside the expression.

Test input: `17 5`

```
Sum:        22
Difference: 12
Product:    85
Quotient:   3
Remainder:  2
Exact:      3.4
```

Now run it with `-17 5`. Quotient: _____ Remainder: _____ Exact: _____

J Time breakdown

[8 min] (8 points)

Task J — one `.cpp` file

Read a whole number of seconds and print it as days, hours, minutes and seconds. Use only `int` variables and the operators `/` and `%`.

Test inputs: `3725` → `3725 s = 0 d 1 h 2 min 5 s`

`200000` → _____

K Shop bill: VAT and change

[13 min] (13 points)

Task K — one `.cpp` file

A stationery shop sells three items. Prices are whole numbers of so'm, so use `int` throughout and store them as constants: `const int NOTEBOOK = 8000, PEN = 2500, CALC = 95000;` and `const int VAT_PERCENT = 12;`

1. Read the three quantities.
2. Print one line per item (quantity and line total), then the subtotal, the VAT and the total. Compute the VAT as `subtotal * VAT_PERCENT / 100`.
3. Read the amount the customer pays and print the change.

Test input: `4 10 1`, then `200000`

```
Notebook   x 4 = 32000
Pen        x 10 = 25000
Calculator x 1 = 95000
Subtotal: 152000 so'm
VAT 12%:  18240 so'm
TOTAL:    170240 so'm
Paid: 200000
Change: 29760 so'm
```

With input `2 3 2` and `250000`: TOTAL _____ Change _____

A classmate writes the VAT as `subtotal * (VAT_PERCENT / 100)`. What VAT does their program print, and why?

L Bonus challenges (up to 10 points, not timed)

Attempt these only after everything above is finished.

1. (3) **The 9/5 trap.** A student converts Celsius to Fahrenheit with `double f = 9 / 5 * c + 32;`. For $c = 25$ what does it give, what should it give, and what is the one-character fix?

2. (3) **Digit sum.** Read a four-digit integer and print the sum of its digits using only `/` and `%`. For `4827` the answer is `21`.

3. (4) **Banknotes.** Read an amount in so'm (an `int`, e.g. `1253500`) and print how many 100 000, 10 000 and 1 000 notes it contains, and the remainder. Expected: `12 x 100000, 5 x 10000, 3 x 1000, remainder 500`.

Submission checklist [5 min]

Before you hand in, confirm each line. Tick the box.

- ☐ Your **name** and **group** are on the front page.
- ☐ Sections A–G are answered **on paper**, before you used a computer to check anything.
- ☐ Every `.cpp` file **builds without errors** and starts with the right `#include` lines.
- ☐ Every variable is **initialised** or read with `cin` before it is used.
- ☐ No division that should be real is done as **integer division**.
- ☐ Every `cin` is preceded by a **prompt**, and every printed value has a **label**.
- ☐ You ran each program with the **test input** and compared it with the expected output.
- ☐ You **raised your hand** and showed the TA this sheet and each program running.

Section	Available	Earned
A Legal identifiers	5	
B Choosing a data type	8	
C Predict the output: arithmetic	12	
D Trace the variables	12	
E Find the mistake	8	
F True or false	7	
G Multiple choice	7	
H A basic informative program	12	
I Integer calculator	8	
J Time breakdown	8	
K Shop bill: VAT and change	13	
Total	**100**	
L Bonus challenges	+10	

Checked by TA: _____

What comes next

- **Lab 4** (the second session this week) explains *why* `1.0 * a / b` works: implicit and explicit type conversion, `static_cast`, and what happens when types are mixed.
- It then adds relational (`<`, `==`) and logical (`&&`, `||`, `!`) operators and turns the `IF ... ELSE` of Lab 2 into real `if/else`.

The habit worth keeping from today: **predict before you run**. If the output surprises you, you have found a gap in your model of C++ — which is the cheapest time to find it.

End of Lab 3